08. Multimodal LLMs - LLaVA & Vision-Language Models 상세 분석

목차

1. Multimodal LLMs 개요
2. LLaVA 아키텍처
3. Vision Encoder (CLIP)
4. Multimodal Projector
5. 성능 측정
6. 트러블슈팅

1. Multimodal LLMs 개요

Multimodal LLMs는 **텍스트와 이미지를 동시에 처리**할 수 있는 모델입니다. vLLM은 LLaVA, Qwen2-VL 등 다양한 multimodal 모델을 지원합니다.

1.1 Multimodal이란?

```
Text-only LLM (Llama):
  Input:  "Describe this"
  Output: "I cannot see images"

Multimodal LLM (LLaVA):
  Input:  "Describe this" + [Image of a cat]
  Output: "This is a photo of an orange cat sitting on a windowsill"
```

1.2 Text-only vs Multimodal

특징	Text-only (Llama)	Multimodal (LLaVA)
Input	Text only	Text + Images
Architecture	Decoder-only	Vision Encoder + Decoder
Components	LLM only	Vision Encoder + Projector + LLM
Use Case	Text generation	Visual Q&A, Image captioning
Model Size	7B params	~8B params (Vision + LLM)
vLLM Support	✅ Full	✅ Full

1.3 vLLM에서 지원하는 Multimodal Models

```
# vLLM supported multimodal models:
- LLaVA (llava-hf/llava-1.5-7b-hf)
- LLaVA-NeXT (llava-hf/llava-v1.6-mistral-7b-hf)
- LLaVA-OneVision (lmms-lab/llava-onevision-qwen2-7b-ov)
- Qwen2-VL (Qwen/Qwen2-VL-7B-Instruct)
- Phi-4-Multimodal (microsoft/Phi-4-multimodal)
- Fuyu (adept/fuyu-8b)
```

주요 차이점:

- **LLaVA**: CLIP vision encoder + Vicuna LLM
- **Qwen2-VL**: Custom vision encoder + Qwen2 LLM
- **Phi-4**: Optimized for efficiency
- **Fuyu**: Simple architecture (no vision encoder)

1.4 Multimodal Architecture 개요

```
┌──────────────────────────────────────────────────┐
│          Multimodal LLM (LLaVA)                  │
├──────────────────────────────────────────────────┤
│                                                  │
│  Input: "Describe this" + [Image]               │
│    │                                             │
│    ├─► Image Processing                          │
│    │     │                                       │
│    │     ├─► Vision Encoder (CLIP)               │
```

```
│    │     │     └─▶ Image Embeddings            │
│    │     │         [batch, 576, 1024]          │
│    │     │                                      │
│    │     └─▶ Multimodal Projector              │
│    │           └─▶ Projected Embeddings        │
│    │               [batch, 576, 4096]          │
│    │                                            │
│    ├─▶ Text Processing                         │
│    │     └─▶ Text Embeddings [batch, seq, 4096]│
│    │                                            │
│    ├─▶ Combine (Image + Text embeddings)      │
│    │     └─▶ [batch, 576+seq, 4096]           │
│    │                                            │
│    └─▶ Language Model (Vicuna)                 │
│          └─▶ Output: "This is a photo of..."   │
│                                                 │
└─────────────────────────────────────────────────┘

vs

┌─────────────────────────────────────────────────┐
│         Text-only LLM (Llama)                   │
├─────────────────────────────────────────────────┤
│                                                 │
│  Input: "Describe this"                        │
│    │                                            │
│    ├─▶ Text Embeddings [batch, seq, 4096]     │
│    │                                            │
│    └─▶ Language Model                          │
│          └─▶ Output: "I cannot see images"    │
│                                                 │
└─────────────────────────────────────────────────┘
```

핵심 차이:

1. **Vision Encoder**: 이미지를 embedding으로 변환
2. **Projector**: Vision embedding을 LLM 차원으로 변환
3. **Combined Input**: 이미지 + 텍스트 embedding 결합

---

2. LLaVA 아키텍처

LLaVA (Large Language and Vision Assistant)는 가장 널리 사용되는 multimodal LLM입니다.

2.1 LLaVA 모델 개요

LLaVA-1.5-7B 구성:

```
LLaVA-1.5-7B:
  - Vision Encoder: CLIP ViT-L/14 (336×336)
  - Projector: 2-layer MLP
  - Language Model: Vicuna-7B

Total params:
  - Vision: ~304M (CLIP ViT-L)
  - Projector: ~8M (MLP)
  - LLM: ~7B (Vicuna)
  - Total: ~7.3B params
```

입출력:

```
Input:
  - Text: "What is in this image?"
  - Image: [3, 336, 336] (RGB image)

Processing:
  1. Image → Vision Encoder → [576, 1024]
  2. [576, 1024] → Projector → [576, 4096]
  3. Text → Tokenizer → [seq_len] → Embedding → [seq_len, 4096]
  4. Combine: [576+seq_len, 4096]
  5. LLM forward → Output tokens

Output:
  "This image shows a cat sitting on a windowsill"
```

2.2 LLaVA Forward Pass

```python
# vllm/model_executor/models/llava.py (simplified)

class LlavaForConditionalGeneration(nn.Module):
    """LLaVA multimodal model."""

    def __init__(self, config: LlavaConfig):
        super().__init__()
```

```
    # ============================================================
    # 1. Vision Encoder (CLIP ViT-L/14)
    # ============================================================
    self.vision_tower = CLIPVisionModel(config.vision_config)

    # ============================================================
    # 2. Multimodal Projector (2-layer MLP)
    # ============================================================
    self.multi_modal_projector = LlavaMultiModalProjector(
        vision_hidden_size=config.vision_config.hidden_size,  # 1024
        text_hidden_size=config.text_config.hidden_size,      # 4096
        projector_hidden_act=config.projector_hidden_act,     # "gelu"
    )

    # ============================================================
    # 3. Language Model (Vicuna-7B)
    # ============================================================
    self.language_model = LlamaForCausalLM(config.text_config)

def forward(
    self,
    input_ids: torch.Tensor,
    positions: torch.Tensor,
    pixel_values: torch.Tensor | None = None,
    image_embeds: torch.Tensor | None = None,
) -> torch.Tensor:
    """
    Args:
        input_ids: [batch, seq_len]  (text tokens)
        pixel_values: [batch, 3, 336, 336]  (images)

    Returns:
        logits: [batch, seq_len, vocab_size]
    """

    # ==============================================================
    # Part A: Vision Processing (if image provided)
    # ==============================================================
    if pixel_values is not None:
        # Step 1: Vision Encoder
        image_features = self.vision_tower(
            pixel_values=pixel_values
        )
        # Shape: [batch, num_patches, vision_hidden_size]
        #      = [batch, 576, 1024]
```

```
        # Step 2: Multimodal Projector
        image_features = self.multi_modal_projector(image_features)
        # Shape: [batch, 576, text_hidden_size]
        #      = [batch, 576, 4096]

    elif image_embeds is not None:
        # Pre-computed image embeddings
        image_features = image_embeds
    else:
        # No image (text-only)
        image_features = None

    # ============================================================
    # Part B: Combine Image + Text
    # ============================================================
    if image_features is not None:
        # vLLM handles merging internally
        # Image embeddings are inserted at <image> token positions
        pass

    # ============================================================
    # Part C: Language Model Forward
    # ============================================================
    outputs = self.language_model(
        input_ids=input_ids,
        positions=positions,
        # Image features merged into input embeddings
    )

    return outputs
```

2.3 Image Token Merging

LLaVA는 **placeholder**를 사용하여 이미지를 텍스트에 삽입합니다.

```
# Input prompt:
prompt = "<image>\nWhat is in this image?"

# Tokenization:
tokens = [
    <image>,  # Special token (replaced by image embeddings)
    \n,
    What, is, in, this, image, ?
```

```
]

# Embedding:
# <image> token → 576 image patch embeddings
# Other tokens → text embeddings

# Final sequence:
embeddings = [
    # Image embeddings (576 patches)
    [image_patch_0],  # [4096]
    [image_patch_1],  # [4096]
    ...
    [image_patch_575], # [4096]

    # Text embeddings
    [\n],     # [4096]
    [What],   # [4096]
    [is],     # [4096]
    ...
]

# Total length: 576 (image) + 8 (text) = 584 tokens
```

코드 예시:

```
# vLLM internally handles <image> token replacement

# User input:
prompt = "<image>\nDescribe this image"
image = PIL.Image.open("cat.jpg")

# vLLM processing:
# 1. Tokenize: [<image>, \n, Describe, this, image]
# 2. Vision encode: image → [576, 1024] → project → [576, 4096]
# 3. Replace <image> with 576 patch embeddings
# 4. Final: [patch_0, ..., patch_575, \n_emb, Describe_emb, ...]
# 5. LLM forward: → "This is a photo of a cat..."
```

---

3. Vision Encoder (CLIP)

LLaVA는 **CLIP Vision Transformer**를 vision encoder로 사용합니다.

3.1 CLIP ViT 구조

```python
# vllm/model_executor/models/clip.py

class CLIPVisionModel(nn.Module):
    """CLIP Vision Transformer for encoding images."""

    def __init__(self, config: CLIPVisionConfig):
        super().__init__()

        # Config:
        # - image_size: 336
        # - patch_size: 14
        # - hidden_size: 1024
        # - num_hidden_layers: 24
        # - num_attention_heads: 16

        # ═══════════════════════════════════════════════════════
        # 1. Vision Embeddings
        # ═══════════════════════════════════════════════════════
        self.embeddings = CLIPVisionEmbeddings(config)

        # ═══════════════════════════════════════════════════════
        # 2. Vision Transformer Encoder (24 layers)
        # ═══════════════════════════════════════════════════════
        self.encoder = CLIPEncoder(config)

        # ═══════════════════════════════════════════════════════
        # 3. Post-layernorm
        # ═══════════════════════════════════════════════════════
        self.post_layernorm = nn.LayerNorm(
            config.hidden_size,
            eps=config.layer_norm_eps
        )

    def forward(self, pixel_values: torch.Tensor) -> torch.Tensor:
        """
        Args:
            pixel_values: [batch, 3, 336, 336]

        Returns:
            image_features: [batch, num_patches, hidden_size]
                            = [batch, 576, 1024]
        """
        # Patch embedding
```

```
        hidden_states = self.embeddings(pixel_values)
        # [batch, 576, 1024]

        # Transformer encoder
        hidden_states = self.encoder(hidden_states)
        # [batch, 576, 1024]

        # Post-layernorm
        hidden_states = self.post_layernorm(hidden_states)
        # [batch, 576, 1024]

        return hidden_states
```

3.2 Image Patching

CLIP은 이미지를 **14×14 patches**로 나눕니다.

```
# Image: 336×336 pixels
# Patch size: 14×14
# Number of patches: (336/14) × (336/14) = 24 × 24 = 576

Image: [3, 336, 336]
  ↓ Split into patches
Patches: [576, 3, 14, 14]
  ↓ Flatten each patch
Patches: [576, 3×14×14] = [576, 588]
  ↓ Linear projection
Patch embeddings: [576, 1024]
```

시각화:

```
Original Image (336×336):
┌──────────────────────────────┐
| Pixel Pixel Pixel ... Pixel |
| Pixel Pixel Pixel ... Pixel |
|  ...                         |
└──────────────────────────────┘

After Patching (24×24 patches):
┌────┬────┬────┬────┬────┐
|P0 |P1 |P2 |...|P23|
├────┼────┼────┼────┼────┤
|P24|P25|...|   |   |
├────┼────┼────┼────┼────┤
```

```
|...|   |   |   |   |
└───┴───┴───┴───┴───┘
Total: 576 patches

Each patch:
  - Size: 14×14 pixels
  - Channels: 3 (RGB)
  - Flattened: 14×14×3 = 588 values
  - Projected: 1024-dim embedding
```

3.3 CLIP Encoder Layer

```python
class CLIPEncoderLayer(nn.Module):
    """Single CLIP Vision Transformer layer."""

    def __init__(self, config: CLIPVisionConfig):
        super().__init__()

        # Self-Attention
        self.self_attn = CLIPAttention(
            embed_dim=config.hidden_size,       # 1024
            num_heads=config.num_attention_heads, # 16
            head_dim=config.hidden_size // config.num_attention_heads,  # 64
        )

        # MLP
        self.mlp = CLIPMLP(
            config.hidden_size,           # 1024
            config.intermediate_size,     # 4096
            config.hidden_act,            # "quick_gelu"
        )

        # LayerNorms
        self.layer_norm1 = nn.LayerNorm(config.hidden_size)
        self.layer_norm2 = nn.LayerNorm(config.hidden_size)

    def forward(self, hidden_states: torch.Tensor) -> torch.Tensor:
        """
        Args:
            hidden_states: [batch, 576, 1024]

        Returns:
            output: [batch, 576, 1024]
        """
```

```
        # Self-Attention
        residual = hidden_states
        hidden_states = self.layer_norm1(hidden_states)
        hidden_states = self.self_attn(hidden_states)
        hidden_states = residual + hidden_states

        # MLP
        residual = hidden_states
        hidden_states = self.layer_norm2(hidden_states)
        hidden_states = self.mlp(hidden_states)
        hidden_states = residual + hidden_states

        return hidden_states
```

---

4. Multimodal Projector

Multimodal Projector는 **vision embedding을 LLM 차원으로 변환**합니다.

4.1 LlavaMultiModalProjector 구조

```
# vllm/model_executor/models/llava.py:118-150

class LlavaMultiModalProjector(nn.Module):
    """2-layer MLP projector for LLaVA."""

    def __init__(
        self,
        vision_hidden_size: int,    # 1024 (CLIP)
        text_hidden_size: int,      # 4096 (Vicuna)
        projector_hidden_act: str,  # "gelu"
    ):
        super().__init__()

        # ═══════════════════════════════════════════════════════
        # Linear 1: vision_hidden_size → text_hidden_size
        # ═══════════════════════════════════════════════════════
        self.linear_1 = ColumnParallelLinear(
            vision_hidden_size,  # 1024
            text_hidden_size,    # 4096
            bias=True,
```

```
        )

        # ═══════════════════════════════════════════════════════
        # Activation: GELU
        # ═══════════════════════════════════════════════════════
        self.act = get_act_fn(projector_hidden_act)  # GELU

        # ═══════════════════════════════════════════════════════
        # Linear 2: text_hidden_size → text_hidden_size
        # ═══════════════════════════════════════════════════════
        self.linear_2 = RowParallelLinear(
            text_hidden_size,    # 4096
            text_hidden_size,    # 4096
            bias=True,
        )

    def forward(self, image_features: torch.Tensor) -> torch.Tensor:
        """
        Args:
            image_features: [batch, 576, 1024]  (from CLIP)

        Returns:
            projected: [batch, 576, 4096]  (LLM dimension)
        """
        # Linear 1: 1024 → 4096
        hidden_states, _ = self.linear_1(image_features)
        # [batch, 576, 4096]

        # GELU activation
        hidden_states = self.act(hidden_states)

        # Linear 2: 4096 → 4096
        hidden_states, _ = self.linear_2(hidden_states)
        # [batch, 576, 4096]

        return hidden_states
```

파라미터 계산:

```
# Linear 1: 1024 → 4096
params_1 = (1024 × 4096) + 4096  # weights + bias
        = 4,194,304 + 4,096
        = 4,198,400

# Linear 2: 4096 → 4096
```

```
params_2 = (4096 × 4096) + 4096
        = 16,777,216 + 4,096
        = 16,781,312

# Total projector params:
total = params_1 + params_2
     = 4,198,400 + 16,781,312
     = 20,979,712 ≈ 21M params

# Very small compared to LLM (7B params)!
```

4.2 Projection 예시

```
# Input (from CLIP):
image_features.shape = [1, 576, 1024]

# Example values (first patch):
image_features[0, 0, :5] = [0.23, -0.45, 0.67, 0.12, -0.34, ...]

# After Linear 1:
hidden = linear_1(image_features)
# [1, 576, 4096]

# After GELU:
hidden = GELU(hidden)

# After Linear 2:
projected = linear_2(hidden)
# [1, 576, 4096]

# Now compatible with LLM input!
# LLM expects: [batch, seq_len, 4096]
```

---

5. 성능 측정

5.1 LLaVA-1.5-7B 성능

```
# Hardware: 1× A100 40GB
```

```
Model: LLaVA-1.5-7B

Throughput:
  - Single image:  ~5 images/sec
  - Batch 4:       ~12 images/sec
  - Batch 8:       ~18 images/sec

Latency:
  - Image encoding: ~50ms (CLIP)
  - Projection:     ~5ms
  - LLM generation: ~150ms (20 tokens)
  - Total:          ~205ms

Memory:
  - Vision Encoder: ~1.2GB (CLIP ViT-L)
  - Projector:      ~0.1GB
  - LLM:            ~14GB (Vicuna-7B FP16)
  - Activations:    ~3GB
  - Total:          ~18GB
```

5.2 최적화 기법

(1) Pre-computed Image Embeddings

```
# Slow: Encode image every time
for request in requests:
    image_features = vision_encoder(request.image)  # 50ms
    output = llm(image_features, request.text)

# Fast: Pre-compute and cache
image_features = vision_encoder(image)  # 50ms once
for request in requests:
    output = llm(image_features, request.text)  # No re-encoding!

# Speedup: 2-3x for multi-turn conversations
```

(2) Batch Processing

```
# Sequential: 4 images × 205ms = 820ms
for image, text in inputs:
    output = llava(image, text)

# Batched: 4 images in one batch = 250ms
```

```
outputs = llava(images_batch, texts_batch)

# Speedup: 3.3x
```

---

6. 트러블슈팅

6.1 Image Size Mismatch

증상:

```
ValueError: Expected image size 336x336, got 512x512
```

원인: CLIP expects fixed size (336×336)

해결:

```
# Resize image before feeding to vLLM
from PIL import Image

image = Image.open("photo.jpg")
image = image.resize((336, 336))  # Resize to expected size

# Or let vLLM handle it automatically (recommended)
```

6.2 OOM with Large Batch

증상: CUDA out of memory with batch size > 4

해결:

```
# 1. Reduce batch size
--max-num-seqs 4 → 2

# 2. Use smaller image resolution
# LLaVA-1.5: 336×336 (default)
# LLaVA-1.6: Can use 672×672 (4x memory!)
```

```
# 3. Quantization
--quantization awq  # Quantize LLM only (not vision encoder)
```

6.3 Slow Image Encoding

증상: High latency (~200ms) for image processing

해결:

```
# 1. Use pre-computed embeddings
# Pass image_embeds instead of pixel_values

# 2. Batch multiple images
# Process multiple requests together

# 3. GPU placement
# Ensure vision encoder on GPU (not CPU!)
```

---

7. 요약

7.1 핵심 포인트

Multimodal LLMs:

1. ✅ **Architecture**: Vision Encoder + Projector + LLM
2. ✅ **CLIP ViT**: 336×336 image → 576 patches → 576×1024 embeddings
3. ✅ **Projector**: 2-layer MLP (1024 → 4096 dimension)
4. ✅ **Integration**: Image embeddings replace tokens

vLLM 최적화:

- Batch processing for images
- Pre-computed embedding caching
- Efficient vision encoder integration

7.2 실무 권장사항

모델 선택:

```
# General purpose: LLaVA-1.5
vllm serve llava-hf/llava-1.5-7b-hf

# Better quality: LLaVA-1.6 (larger images)
vllm serve llava-hf/llava-v1.6-mistral-7b-hf

# Best performance: Qwen2-VL
vllm serve Qwen/Qwen2-VL-7B-Instruct
```

배포 설정:

```
# Standard deployment:
vllm serve llava-hf/llava-1.5-7b-hf \
  --max-num-seqs 4 \
  --gpu-memory-utilization 0.85

# High throughput:
vllm serve llava-hf/llava-1.5-7b-hf \
  --max-num-seqs 8 \
  --enforce-eager  # Disable CUDA graphs
```

7.3 참고 자료

논문:

- LLaVA: "Visual Instruction Tuning" (2023)
- CLIP: "Learning Transferable Visual Models From Natural Language Supervision" (2021)

vLLM 코드:

- LLaVA: `vllm/model_executor/models/llava.py`
- CLIP: `vllm/model_executor/models/clip.py`
- Vision utils: `vllm/model_executor/models/vision.py`

관련 문서:

- 01. Weight Loading
- 02. Model Initialization
- 05. Transformer LLMs

문서 작성 완료! 🎉

이 문서에서 다룬 내용:

1. Multimodal LLMs 개요 (vs Text-only)
2. LLaVA 아키텍처 (Vision Encoder + Projector + LLM)
3. CLIP Vision Transformer (Image patching, ViT encoder)
4. Multimodal Projector (2-layer MLP, 1024→4096 projection)
5. 성능 측정 및 최적화 기법
6. 트러블슈팅 가이드

Multimodal LLMs의 핵심 개념을 vLLM 코드 기반으로 상세히 분석했습니다!